Accelerated array computing and code transformations for R

Sebastian Fischer[1] · Daniel Falbel · Martin Binder[1] · Bernd Bischl[1]

[1]LMU Munich, Munich Center for Machine Learning (MCML)

Motivation

anvl brings **compiled, GPU-accelerated array computing** to the R community. You write array programs in plain R, which anvl compiles under the hood to optimized machine code via the **OpenXLA** compiler stack — the same infrastructure behind JAX, TensorFlow, and PyTorch/XLA. In short, anvl aspires to be for R what JAX is for Python.

① Execution: eager & JIT

Eager mode (default): each operation runs immediately — ideal for interactive exploration and debugging.

JIT mode: `jit()` compiles a whole function into one fused executable. XLA optimizes across operations, which is where the speedups come from.

```
y    <- predict(X, beta)  # eager: runs now
fast <- jit(predict)       # compile
y    <- fast(X, beta)      # fast
```

② Automatic differentiation

`gradient()` returns a new function that computes **reverse-mode gradients** of a plain R function — no derivations by hand. It composes with `jit()`, so a loss, its gradient, and a training loop are all plain R.

```
grad <- gradient(loss, wrt = "beta")
g    <- grad(X, y, beta)
```

What anvl can do

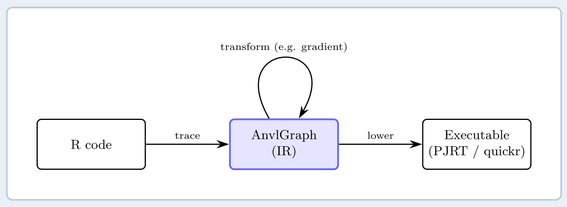
≈ 140 array operations. Construction, arithmetic, linear algebra (`nv_matmul`, `nv_qr`, `nv_svd`, `nv_chol`, `nv_solve`), reductions and statistics, random-number generation, and JIT-able control flow (`nv_while`).

Build, in plain R. Neural networks, matrix factorizations, custom optimizers, gradient-based estimators, and Monte Carlo samplers — all expressed directly in R and run on CPU or GPU.

Why anvl? GPU acceleration without leaving R · simplified CUDA install (only an NVIDIA driver needed) · JIT compilation sets it apart from **{torch}** · written primarily in R, so easy to extend and contribute to.

⚠️ **Scope:** anvl currently targets **dense** arrays — sparse matrices are not well supported yet. Best fit: dense data, regular shapes, and repeated or batched computation.

How it works



Trace R code → AnvlGraph (IR) · **Transform** graph → graph (e.g. `gradient()`) · **Lower** graph → device executable (via OpenXLA)

★ Application: randomized SVD for bioinformatics

Randomized SVD approximates the top-*k* singular triplets of a large matrix by first sketching it onto a low-dimensional subspace with a random Gaussian matrix Ω, then doing an exact SVD of the small projection. It is a matmul-heavy, dense workload — exactly XLA's sweet spot — so `jit()`-compiling it on a GPU scales these decompositions to larger data.

```
rsvd <- jit(function(A, Omega) {
  Q <- nv_qr(nv_matmul(A, Omega))$Q   # sketch + orthonormalize
  B <- nv_matmul(nv_transpose(Q), A)  # project (small)
  s <- nv_svd(B)                      # exact SVD of B
  list(d = s$d, u = nv_matmul(Q, s$u), vt = s$vt)
})
```

Benchmark figure

Placeholder — runtime of randomized SVD
(base R vs. anvl CPU vs. anvl GPU)
to be added once data is available.

⚠️ **For Bioconductor users:** sparse single-cell / genomics matrices would need densifying first — native sparse support is future work.

Getting started

Install from r-universe — CPU works out of the box; NVIDIA GPU support is added via the `cuda12.8` package.

Runs on: CPU — Linux, Windows, Apple Silicon · NVIDIA GPU — Linux x86-64.

Status: early-stage, not yet on CRAN — feedback and contributions welcome.

github.com/r-xla/anvl **r-xla.github.io/anvl**

Supported by **MaRDI** — the Mathematical Research Data Initiative, an NFDI project funded by DFG grant 460135501.